



SR UNIVERSITY
Campus Warangal

Program: II - B. Tech (CS& AI) Batch: 58

Name: Ch.snehitha
No: 2503A54035

Reg.

Department: Computer Science and AI Semester: I

AI Assisted Coding-Assignment Date: 20-09-2026

1Q : Class with Conditional Logic – Employee Salary .

Prompt : Create a Python program using an Employee class with name and basic_salary. Calculate a 10% bonus if salary $\geq$ 50,000, otherwise 5%. Calculate and display the total salary. Create two employee objects and test both conditions.

Code : # Employee Salary Calculator

```
class Employee:
```

```
    def __init__(self, name, basic_salary):  
        self.name = name  
        self.basic_salary = basic_salary
```

```
    # Calculate bonus based on salary
```

```
    def calculate_bonus(self):  
        if self.basic_salary >= 50000:  
            return self.basic_salary * 0.10  
        else:  
            return self.basic_salary * 0.05
```

```
    # Calculate total salary
```

```
    def calculate_total_salary(self):  
        return self.basic_salary + self.calculate_bonus()
```

```
    # Display employee details
```

```
    def display_details(self):  
        bonus = self.calculate_bonus()  
        total_salary = self.calculate_total_salary()
```

```
    print("Employee Name:", self.name)  
    print("Basic Salary: ₹", self.basic_salary)  
    print("Bonus: ₹", bonus)  
    print("Total Salary: ₹", total_salary)  
    print("-" * 30)
```

```
# Create two employee objects
```

```
employee1 = Employee("Rahul", 60000)  
employee2 = Employee("Priya", 40000)
```

```
# Display details
```

```
employee1.display_details()
```

```
employee2.display_details()
```

Output:

```
PS C:\Users\CH.VIVEK REDDY\OneDrive\Documents\Desktop\Ai_assisted assignment> & "C:\Users\CH.VIVEK REDDY\OneDrive\Documents\Desktop\Ai_assisted assignment/Q1_employee_salary.py"
• " "C:/Users/CH.VIVEK REDDY/OneDrive/Documents/Desktop/Ai_assisted assignment/Q1_employee_salary.py"
Employee Name: Rahul
Basic Salary: ₹ 60000
Bonus: ₹ 6000.0
Total Salary: ₹ 66000.0
-----
Employee Name: Priya
Basic Salary: ₹ 40000
Bonus: ₹ 2000.0
Total Salary: ₹ 42000.0
-----
```

Observation The Employee class was created successfully using conditional logic. The program calculates a 10% bonus for employees whose basic salary is ₹50,000 or above and a 5% bonus for salaries below ₹50,000. The total salary and employee details are displayed successfully.

2Q: Loop and Conditional Logic – Student Marks

Prompt: Write a Python program that accepts a list of student marks and uses a loop to process each mark. Display "Distinction" for marks ≥ 75 , "Pass" for marks ≥ 40 and < 75 , and "Fail" for marks < 40 . Ignore invalid marks outside the range 0–100 and calculate and display the number of students in each category. Test the program with a suitable list of marks.

```
Code : # Student Marks Analyzer
```

```
# List of student marks
marks = [85, 72, 45, 30, 95, 60, 110, -5]
```

```
# Counters for each category
distinction_count = 0
pass_count = 0
fail_count = 0
```

```
# Process each mark using a loop
for mark in marks:
```

```
    # Ignore invalid marks
    if mark < 0 or mark > 100:
        print("Invalid mark ignored:", mark)
        continue
```

```
    # Categorize the valid marks
    if mark >= 75:
        print(mark, "- Distinction")
        distinction_count += 1
```

```
    elif mark >= 40:
```

```
print(mark, "- Pass")
pass_count += 1
```

```
else:
    print(mark, "- Fail")
    fail_count += 1
```

```
# Display the number of students in each category
print("\nNumber of Distinction students:", distinction_count)
print("Number of Pass students:", pass_count)
print("Number of Fail students:", fail_count)
```

Output :

```
● P& "C:\Users\CH.VIVEK REDDY\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users
Q2_student_marks.py"
85 - Distinction
72 - Pass
45 - Pass
30 - Fail
95 - Distinction
60 - Pass
Invalid mark ignored: 110
Invalid mark ignored: -5

Number of Distinction students: 2
Number of Pass students: 3
Number of Fail students: 1
```

Obsevation :The program successfully processes the list of student marks using a loop. It classifies students as Distinction, Pass, or Fail based on their marks, ignores invalid marks outside the range 0–100, and displays the count of students in each category.

3Q : **Class + Loop – Product Inventory**

Prompt : Create a Python program using a Product class that stores product name, price, and quantity. Create a method total_value() to calculate price × quantity. Create a list of multiple Product objects and use a loop to calculate the total inventory value. Display the products whose quantity is less than 5 as "Low Stock". Test the program with at least three products.

Code : # Product Inventory Management

```
class Product:
    def __init__(self, name, price, quantity):
        self.name = name
        self.price = price
        self.quantity = quantity
```

```
# Calculate the total value of this product
```

```
def total_value(self):  
    return self.price * self.quantity
```

```
# Create a list of Product objects  
products = [  
    Product("Laptop", 50000, 3),  
    Product("Mouse", 800, 10),  
    Product("Keyboard", 1500, 4)  
]
```

```
# Calculate total inventory value  
total_inventory_value = 0
```

```
print("Product Inventory")  
print("-" * 40)
```

```
for product in products:  
    value = product.total_value()  
    total_inventory_value += value
```

```
print("Product Name:", product.name)  
print("Price: ₹", product.price)  
print("Quantity:", product.quantity)  
print("Total Value: ₹", value)
```

```
# Check for low stock  
if product.quantity < 5:  
    print("Status: Low Stock")
```

```
print("-" * 40)
```

```
# Display total inventory value  
print("Total Inventory Value: ₹", total_inventory_value)
```

Output :

```

Number of Fall students: 1
● PS C:\Users\CH.VIVEK REDDY\OneDrive\Documents\Desktop\Ai_assisted assignment> & "C:\Users\CH.VIVEK REDDY\AppData\Local\
" "c:/Users/CH.VIVEK REDDY/OneDrive/Documents/Desktop/Ai_assisted assignment/Q3_product_inventory.py"
Product Inventory
-----
Product Name: Laptop
Price: ₹ 50000
Quantity: 3
Total Value: ₹ 150000
Status: Low Stock
-----
Product Name: Mouse
Price: ₹ 800
Quantity: 10
Total Value: ₹ 8000
-----
Product Name: Keyboard
Price: ₹ 1500
Quantity: 4
Total Value: ₹ 6000
Status: Low Stock
-----
Total Inventory Value: ₹ 164000

```

Observation : The Product class was created successfully to store product name, price, and quantity. The program calculates the total value of each product using price × quantity, calculates the total inventory value using a loop, and identifies products with quantity less than 5 as "Low Stock."

4Q. Class + Loop + Nested Conditions – Electricity Bill

Prompt : Create a Python program using a class named ElectricityBill that stores the consumer name and units consumed. Calculate the electricity bill using the following rates: first 100 units at ₹2 per unit, 101–200 units at ₹3 per unit, and above 200 units at ₹5 per unit. Use appropriate conditional statements and loops where required. Create at least three objects with different unit values and display the consumer name, units consumed, and total bill. Test the program with different unit values, including an edge value such as 100 or 200.

Code : # Electricity Bill Calculator

```

class ElectricityBill:
    def __init__(self, consumer_name, units):
        self.consumer_name = consumer_name
        self.units = units

```

```

# Calculate electricity bill
def calculate_bill(self):
    units = self.units

```

```

if units <= 100:
    bill = units * 2

```

```

elif units <= 200:
    bill = (100 * 2) + ((units - 100) * 3)

```

```

else:
    bill = (100 * 2) + (100 * 3) + ((units - 200) * 5)

```

```
return bill
```

```
# Display bill details
def display_bill(self):
    total_bill = self.calculate_bill()
```

```
print("Consumer Name:", self.consumer_name)
print("Units Consumed:", self.units)
print("Total Bill: ₹", total_bill)
print("-" * 35)
```

```
# Create three ElectricityBill objects
consumer1 = ElectricityBill("Rahul", 100)
consumer2 = ElectricityBill("Priya", 200)
consumer3 = ElectricityBill("Anil", 250)
```

```
# Display bills
consumer1.display_bill()
consumer2.display_bill()
consumer3.display_bill()
```

Output :

```
-----
● PS C:\Users\CH.VIVEK REDDY\OneDrive\Documents\Desktop\Ai_assisted assignment> & "C:\Users\CH.VIVEK REDDY\OneDrive\Documents\Desktop\Ai_assisted assignment\Q4_electricity_bill.py"
Consumer Name: Rahul
Units Consumed: 100
Total Bill: ₹ 200
-----
Consumer Name: Priya
Units Consumed: 200
Total Bill: ₹ 500
-----
Consumer Name: Anil
Units Consumed: 250
Total Bill: ₹ 750
-----
○ PS C:\Users\CH.VIVEK REDDY\OneDrive\Documents\Desktop\Ai_assisted assignment> █
```

Observation : The ElectricityBill class was created successfully to store the consumer name and units consumed. The program calculates the electricity bill based on the given unit ranges and displays the consumer name, units consumed, and total bill. Different unit values, including edge values such as 100 and 200, were tested successfully.